

Project SLIME ROLLER

Member Names: Joshua Kim-Pearson, Stanley Hung, Edison Chan, Brody Vance, Tristan Chen, Lorenzo Uk

Emails: jkimpear@ucsc.edu, stchhung@ucsc.edu, echan50@ucsc.edu, blvance@ucsc.edu, trzchen@ucsc.edu, luk@ucsc.edu

Githubs: JoshuWoshua1, Stan-21, Edison272, lorenzouk, TristanChenUCSC, blvance

Project: Final, built on marble physics game (Assignment 2)

- However, this game also uses stealth mechanics from Assignment 4

Summary of Changes: We added an ability system to our rolling slime which can change how the player behaves. By default the player can charge a jump, and by using abilities they can solve unique puzzles in each level. Additionally we added 4 new levels and a new hub zone for the player to explore and beat. New shaders and particles were added to the project as well, some from our assignment 3 but others were made fresh for this project. We also got an asset pack for the environment and got audio as well.

Graphics Integration:

Lightning:

- Rooms: Simple directional lights and point lights create a dark atmosphere illuminated by artificial lights
- Drones & Cameras: Drones and Cameras have lights to represent the different states their AI is in. Yellow for Patrolling, Red for Actively Attacking, and Blue for Stunned
- Slime: Each Slime Powerup/State causes the slime to emit a different colored light. Green for Basic/Jump, Red for Chrome, Purple for Gravity, Blue for Electric Shock, and no light for Shadow

Shaders:

- Laser shader: For the lasers, I made a simple shader that scrolls a texture left to right and emits a bright HDR color to give off a glowing / shooting effect.
- SlimeMembrane "Container" shader: Uses a normal map + metallic reflection in order to define the container's shape despite being fully transparent.
- SlimeCore "Goo" shader: Simulates basic liquid physics. Gives the object's material a "fill" level like filling water into a cup, by setting transparency based on the pixel's object position. Uses a custom script combined with rotational offsets on the x & z axis to simulate the a "liquid wobbling" effect whenever the object moves
- ShockCore shader: Applies a custom weighted noisemap to the positions of the model's pixel positions to create a "spiky" effect. Using a shifting tilemap causes the spikes to flicker like electric sparks. HDR Colors and fresnel effect allow the material to glow.

Particles:

- Spark Particles: used in the EMP effect of the shock slime as well as the broken doors scene across various levels

- Shade Particles: used by the Shade/Shadow/Ninja slime whenever they disappear and reappear from sight

Audio Assets

Sound Effects:

[My Instants](#), [Pixabay](#), Roblox, Star Wars

Music Used (If you can hear it)

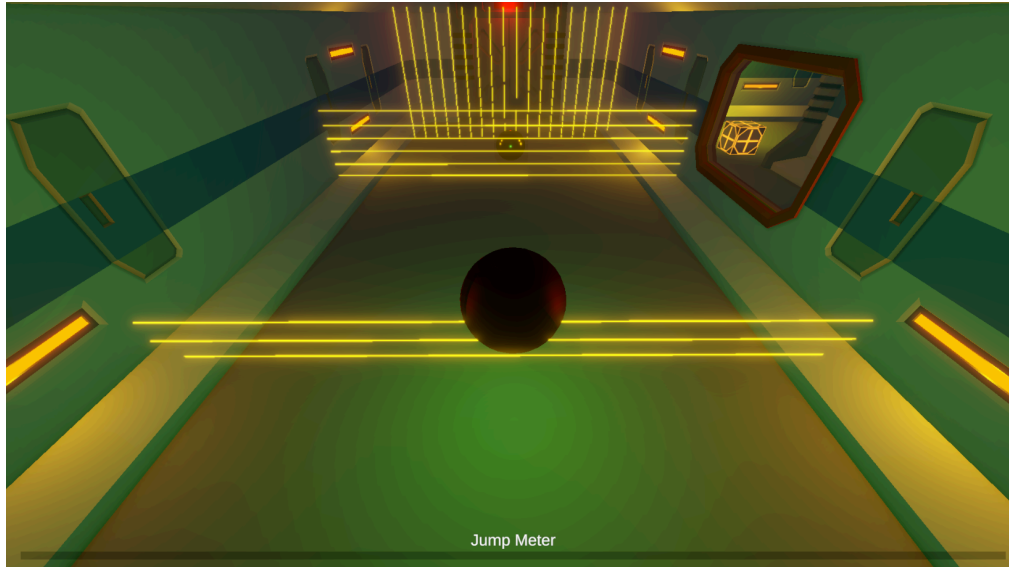
[Irving Heat - Stealth Music Beats](#)

[FilFar - Bitrack](#)

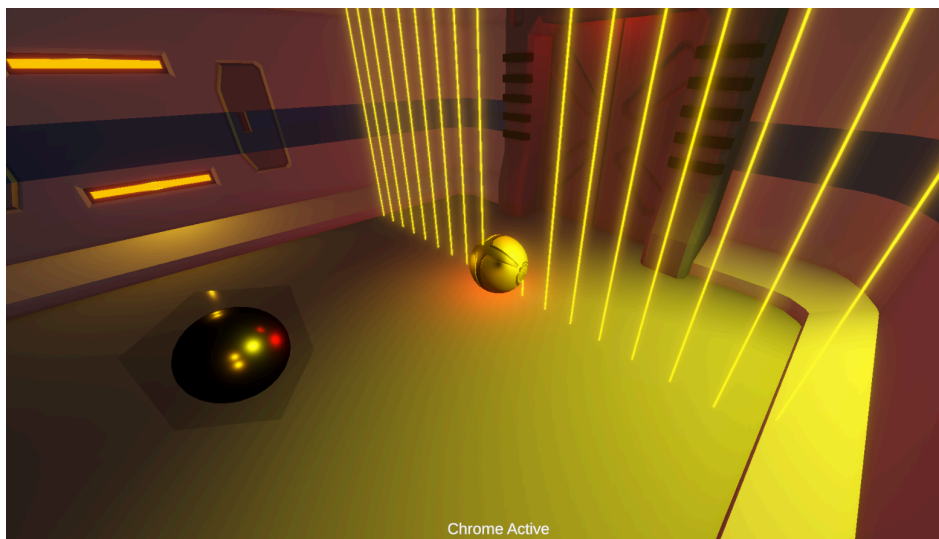
Screenshots



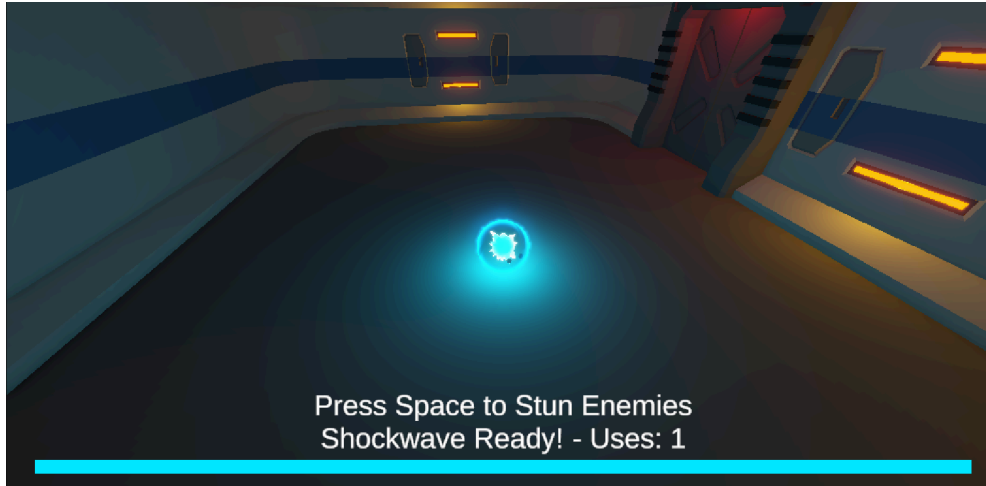
Shadow ability is being used to sneak under a small gap



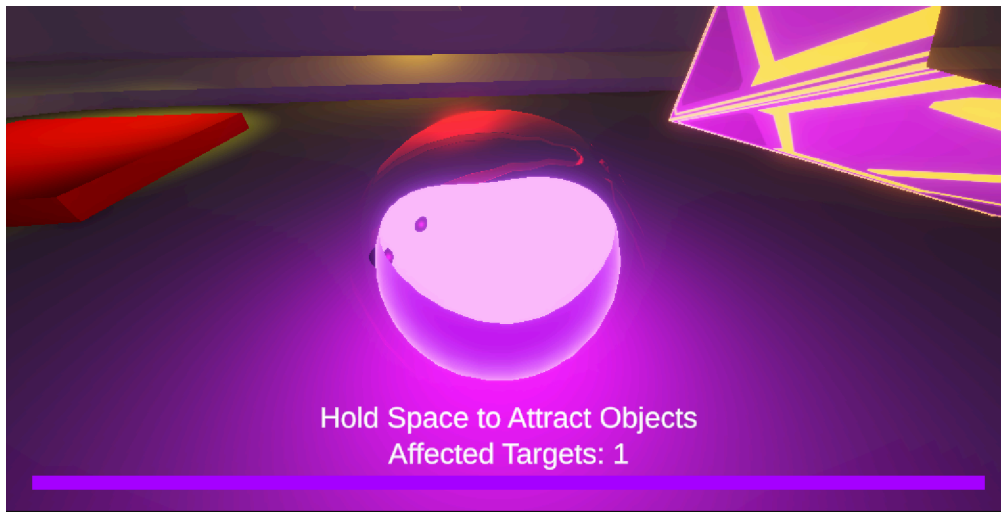
Jump is being used to clear lasers on the floor



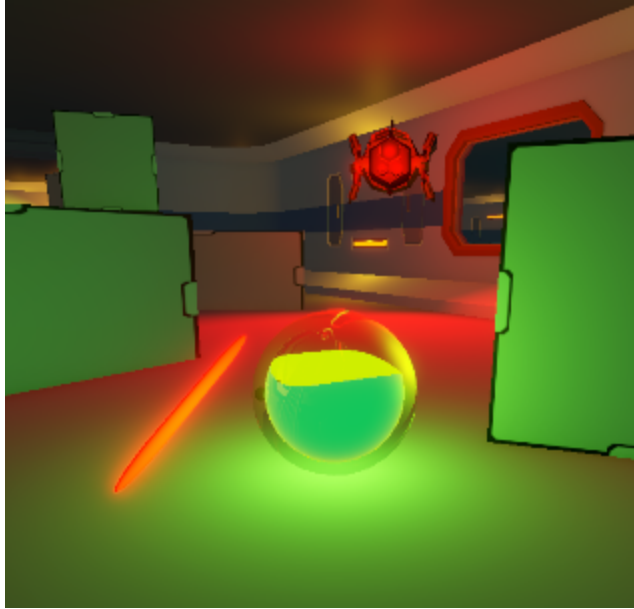
Chrome ability pickup & chrome slime being able to roll through lasers freely.



Shockwave ability could be used to stun enemies temporarily.



Gravity ability allows the player to attract objects to themselves by holding space.



Drones Shoot at the player!

Reflections

Josh Reflection: I created our hub scene where the player may move around and load into levels. In between the hub and levels, I load a quick loading screen to show where the player is going. I additionally handled the scene transitions and the shadow player ability. I created a few particle effects and utilized a projector for the shadow form, and it changes the tag of the player to make it invisible to certain hazards like the drones and cameras, and ignores collisions with certain objects to give the illusion of slipping under them as a shadow. One technical challenge I overcame was toggling collisions based on tags with the shadow ability. I also created the Main Menu, settings, and some UI elements as well.

The most impactful things I've learned this quarter would be ShaderGraph and ProBuilder. As I've gotten more comfortable with these tools, game development has become easier. I found myself using these a lot this quarter in CMPM170 for our prototypes, and it quickened my progress on those projects. One more thing that I

want to explore more from this quarter is the observer pattern. I remember the lecture on it, but I haven't had a good opportunity to use it myself yet, so I plan on messing around with it this summer. If I had more time for this project, I would have put more effort into shaders and general polish, especially since most of my time these past 2 weeks has been focused on CMPM170.

Stan Reflection: I created the laser beam, pressure plate, and the chrome module for the game. The laser beam had two different variations, one that would shoot a fixed distance, and one that would shoot from point A to point B. Both of these lasers had the same reflection mechanic, so if they were colliding with a reflective material, it would reflect at a certain angle. The chrome module went hand in hand with the laser obstacles as they were the main form of immunity against them. The pressure plate was a simple trigger collider that would call an "open" and "close" function on the objects that it was connected to.

The main technical challenges that I had were figuring out the laser. I had to go more in-depth with how Raycasts and LineRenderers work, and figure out how to calculate the reflection angle. Even though a lot of the heavy math portions were already handled in the engine, it was still challenging for me to figure it all out and put it together to get the laser system working.

Throughout the quarter, I learned a lot about Unity. My prior experience was just getting thrown into a project so I did have some experience, but I did not have a foundation in Unity when working there so this class definitely built that foundation. The biggest thing that I learned in this class was using Shadergraph. If I had more time to work on this project, I definitely would've contributed more to the project, especially in the ideation parts. Unfortunately, I was very swamped with CMPM 172, so I mainly just worked on what I was assigned to do. There's a lot of cool mechanics that could be implemented in this game and so much room for polish with shaders, lighting, and particles that I could've also worked on if I had more time.

Edison Reflection: Some of my achievements include shaders/VFX

for various abilities, the functionality for the Jump & Shock Modules, and the underlying architecture for the ability system. I made three main shaders: one “SlimeMembrane” shader, a green “SlimeCore” shader, and an electricity shader. My greatest technical achievement might have to be the ability system I made, to solve the problem of dealing with multiple diverse abilities. To make creating abilities easy and consistent, I devised a basic architecture for how all abilities are made, which uses Scriptable Objects to hold data and instantiate everything, a player module class that interacts with the player and holds local data, and custom UI. Any time a new ability is made, simply make all three scripts. I used this system to make the Jump and Shock Modules.

I tend to work alone in Unity, so I’ve never had to worry about coordinating my efforts with other people. So I guess a “minus” about working in a team where everyone is actually contributing is that I can no longer work wherever I want in the project, and I’ve gained a valuable amount of experience on how to plan out a project from start to finish, and maintaining scope within a project to reduce the amount of conflicts and accidental scene wipes which occur.

If I had more time and was willing to do work on this earlier, The biggest thing I would’ve wanted to do is to try implementing more organization into our workflow. We effectively started and finished this assignment in 2-3 days, due to all of us prioritizing classes. Our best attempt at organizing work was making a checklist of tasks and blowing up the group chat with questions and progress. That, and also maybe playtest our game because I’ve heard it’s insanely difficult.

Brody Reflection: I created the shader and vfx graph for the blackhole slime that was used to display the gravity mode pick-up. I also made prefabs for the ability collectibles. I also coded the ability switching through picking up the actual collectible. A technical challenge I overcame was on my black hole shader. I was having an issue with scale and lighting

affecting my shader in unexpected ways. I overcame this by adjusting the shader graph nodes' numbers and calculations to work independently from the object scale and by refining the lighting setup conditions. Through some testing I was able to achieve a more stable and visually consistent black hole effect. This process of testing gave me more insight on how shaders interact with other things inside scenes.

Something I learned this quarter about Unity in general is that it is very versatile in what it can output. It gives a multitude of tools at your disposal to create many different unique effects and gameplay feelings. The physics system creates player interactions with materials and movement, shaders and visual effects can improve the look and feel of a game while not ruining performance because it goes to the GPU, and learning how UV mapping works was so interesting because I never even knew that's how pathing worked in many other games. I also became more comfortable troubleshooting technical problems and bugs using Unity's toolset to experiment with different solutions.

If I had more time I would learn more about the shaders as the final project was where I started to learn since I previously did particles. Shaders are so useful and cool that I wish I put more time into learning the ins-and-outs of them. With that said I would further polish my black hole effect to be more complete. With more time I would also like to better optimize the systems that I created as I feel like there is always room for improvement and tips and tricks that I still don't know about in Unity's vast array of systems.

Tristan Reflection: My main contributions include creating the gravity module and building the AI for the drones/security cameras. I designed the Finite State Machines for both the drone AI and security camera AI, each consisting of 2 states: Patrol/Scanning and Attack. I also helped with making a stun function for the drones and cameras so that they can interact with the shock module. Some challenges I faced while working on these features include trying to understand and build off from another teammate's code. For example, Edison designed the code

architecture for the Player Modules and their scriptable objects. Because he was more experienced than me, I had a hard time implementing the gravity module at first because I was trying to understand all these new concepts, like scriptable objects and input action callback context.

Overall, I was able to learn a lot from this project. Creating the gravity module and trying to understand the code structure for the module system has taught me a lot about abstract classes and how entire systems can be built upon these foundational scripts and inheritance. I also learned about scriptable objects and how they can be used to store and initialize data types. Designing the drones and security cameras also improved my understanding and ability to build FSMs. If I had more time to work on this project, I would've loved to explore Shader Graph more since I didn't get much experience with creating shaders.

Lorenzo Reflexion: I focused primarily on level design and world building of the project. I designed the level layout and played around with lighting during the project. I designed pressure plate switches and doors for the player to navigate around and put every feature implemented all together to create a fun gameplay experience.

A technical challenge that I have encountered was game design itself. I had to ask myself the question, How would I design the game for the best experience without making players feel frustrated or turned away from the game. I had to figure out the best way to utilize each feature we created and figure out how to design an immersive experience that not only challenges the player but gives them a fun gameplay experience.

Throughout this quarter, I have learned and practiced fine tuning my skills in unity. Unity has many tools at its disposal that I have learned and practiced such as NavMeshs, Physics, & Shaders. With more time I would have wanted to practice more on shaders. I don't really know much about them and I wish I had more time to invest in creating them.